



CNO IV Programación WAP

NO PRESIONES ESC... TODAVÍA



Monica Isabel Lopez Alcocer - 179827



24 / 08 / 2026



1. INTRODUCCIÓN



El programa desarrollado consiste en un sistema de **registro de eventos del teclado** utilizando **Python** y la librería **pynput**.



Su función principal es **detectar las acciones** realizadas durante la ejecución del programa, registrar cuándo una tecla es **presionada** o **liberada**, agregar una marca de **fecha y hora**, y almacenar la información en un archivo local llamado **Eventos.txt**.



El objetivo principal es implementar la **persistencia** de registros y la **asociación temporal** de cada evento generado.



Act2_Ware.py

```
1  from pynput.keyboard import Key, Listener
2  # Importa la función datetime para obtener la fecha y hora actual
3  from datetime import datetime
4  # Variable global que almacena la cantidad de eventos registrados
5  contador_eventos = 0
6
7  # Crea un identificador único y consecutivo para cada evento registrado en el archiv
8  def generar_evento():
9      global contador_eventos
10
11     # Aumenta el contador cada vez que ocurre un evento
12     contador_eventos += 1
13     # Devuelve el identificador con tres dígitos
14     return f"evento_{contador_eventos:03d}"
15
16 # Guarda la información del evento ocurrido dentro de un archivo de texto local
17 def guardar_evento(tipo_evento):
18
19     # Obtiene la fecha y hora actual
20     fecha_hora = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
21
22     # Genera un identificador para este evento
23     evento = generar_evento()
24
25     # Abre el archivo y agrega información sin borrar datos previos
26     with open("Eventos.txt", "a", encoding="utf-8") as archivo:
27         # Escribe la información del evento en una nueva línea
28         archivo.write(f"{fecha_hora} | {tipo_evento} | {evento}\n")
29
30 # Se ejecuta automáticamente cuando se presiona una tecla.
31 def on_press(key):
32     try:
33         # Si es una tecla alfanumérica muestra el carácter
34         print(f"Tecla alfanumérica presionada: {key.char}")
```




2. LIBRERÍAS Y CLASES UTILIZADAS

- Para desarrollar el programa se utilizaron las siguientes librerías:



pynput.keyboard

Permite detectar eventos del teclado mediante dos elementos principales:

- **Key:** Identifica teclas especiales como ESC, ENTER o SHIFT.
- **Listener:** Mantiene activo el monitoreo del teclado y ejecuta funciones automáticamente cuando ocurre un evento.



datetime

Permite obtener la fecha y hora actual del sistema para asociarla a cada evento registrado.



Act2_Ware.py

```
1  # Importa las clases necesarias de pynput
   # para detectar eventos del teclado

2  from pynput.keyboard import Key, Listener

3  # Importa la función datetime para obtener
   # la fecha y hora actual

4  from datetime import datetime
```




3. FUNCIÓN GENERAR_EVENTO()

La función `generar_evento()` se encarga de crear un **identificador único** para cada evento registrado.

SU FUNCIONAMIENTO CONSISTE EN:



1

Incrementar el contador de eventos.

123

2

Generar un nombre consecutivo.

000

3

Devolver el identificador con un formato de tres dígitos.



Act2_Ware.py

```
1  # Crea un identificador único y consecutivo
2  # para cada evento registrado en el teclado.
3  def generar_evento():
4      global contador_eventos
5
6      # Aumenta el contador cada vez que
7      # ocurre un evento
8      contador_eventos += 1
9
10     # Devuelve el identificador
11     # con tres dígitos
12     return f"evento_{contador_eventos:03d}"
13
14
15
16
```




4. FUNCIÓN GUARDAR_EVENTO()



La función `guardar_evento()` es la encargada de almacenar la información generada en un archivo de texto.



Cada registro contiene:

- Fecha y hora del evento.
- Tipo de evento generado.
- Identificador único.



Los datos son almacenados en el archivo:

Eventos.txt



Act2_Ware.py

```
1  # Guarda la información del evento ocurrido dentro de un
2  archivo de texto llamado "Eventos.txt".
3  def guardar_evento(tipo_evento):
4      # Obtiene la fecha y hora actual
5      fecha_hora = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
6
7      # Genera un identificador para este evento
8      evento = generar_evento()
9
10     # Abre el archivo y agrega información sin borrar datos previos
11     with open("Eventos.txt", "a", encoding="utf-8") as archivo:
12         # Escribe la información del evento en una nueva línea
13         archivo.write(f"{fecha_hora} | {tipo_evento} | {evento}\n")
```




5. FUNCIÓN ON_PRESS()



La función `on_press()` se ejecuta automáticamente cuando una tecla es presionada.



SU FUNCIONAMIENTO ES:



Detectar si la tecla es alfanumérica.



Identificar teclas especiales.



Mostrar la información en consola.



Enviar el evento a la función `guardar_evento()`.



EL EVENTO GENERADO SE REGISTRA COMO:

PRESS

Indicando que una tecla fue presionada.



Act2_Ware.py

```
1  # Se ejecuta automáticamente cuando se presiona una tecla.
2  def on_press(key):
3
4      try:
5          # Si es una tecla alfanumérica muestra el carácter
6          print(f"Tecla alfanumérica presionada: {key.char}")
7      }
8
9      except AttributeError:
10         # Si es una tecla especial
11         print(f"Tecla especial presionada: {key}")
12
13     # Guarda el evento de tipo PRESS en el archivo
14     guardar_evento("PRESS")
15
16
```




6. FUNCIÓN ON_RELEASE()



La función `on_release()` se ejecuta cuando una tecla deja de estar presionada.



SUS FUNCIONES PRINCIPALES SON:

- ✓ Mostrar la tecla liberada.
- ✓ Registrar el evento dentro del archivo.
- ✓ Finalizar la ejecución cuando se detecta la tecla ESC.



EL EVENTO REGISTRADO SE IDENTIFICA COMO:

RELEASE

Permitiendo diferenciar entre presionar y soltar una tecla.



Act2_Ware.py

```
1  # Se ejecuta automáticamente cuando una tecla deja de estar presionada.
2  def on_release(key):
3      # Muestra en consola la tecla liberada
4      print(f"Tecla liberada: {key}")
5
6      # Guarda el evento de tipo RELEASE en el archivo
7      guardar_evento("RELEASE")
8
9      # Si la tecla liberada es ESC finaliza el programa
10     if key == Key.esc:
11         return False
```


7. FUNCIONAMIENTO DEL LISTENER Y ALMACENAMIENTO DE INFORMACIÓN



PROCESO DE EJECUCIÓN

- 1 El Listener comienza a escuchar eventos del teclado.
- 2 Cuando se presiona una tecla, ejecuta on_press().
- 3 Cuando se libera una tecla, ejecuta on_release().
- 4 Ambas funciones llaman a guardar_evento().
- 5 La información se almacena automáticamente en Eventos.txt.



El Listener funciona como el controlador principal del programa.



EVIDENCIA DE EJECUCIÓN EN CONSOLA

```
PS C:\Users\Monica\AppData\Local\Programs\Microsoft V
Tecla alfanumérica presionada: d
Tecla liberada: 'd'
Tecla alfanumérica presionada: a
Tecla liberada: 'a'
Tecla alfanumérica presionada: o
Tecla liberada: 'o'
Tecla alfanumérica presionada: q
Tecla liberada: 'q'
Tecla alfanumérica presionada: `
Tecla liberada: ['`']
Tecla alfanumérica presionada: q
Tecla alfanumérica presionada: q
Tecla alfanumérica presionada: q
Tecla alfanumérica presionada: q
Tecla alfanumérica presionada: q
Tecla alfanumérica presionada: q
Tecla alfanumérica presionada: q
Tecla alfanumérica presionada: u
Tecla liberada: 'u'
```

```
# # Listener mantiene activo el monitoreo del teclado y llama automáticamente a las funciones
with Listener(
    on_press=on_press,
    on_release=on_release
) as listener:

    # Mantiene el programa ejecutándose esperando eventos
    listener.join()
```



EVIDENCIA DEL ARCHIVO EVENTOS.TXT

Eventos.txt			
Archivo	Editar	Ver	
2026-08-21 21:33:03	PRESS	evento_001	
2026-08-21 21:33:03	RELEASE	evento_002	
2026-08-21 21:33:03	PRESS	evento_003	
2026-08-21 21:33:04	RELEASE	evento_004	
2026-08-21 21:33:04	PRESS	evento_005	
2026-08-21 21:33:04	RELEASE	evento_006	
2026-08-21 21:33:04	PRESS	evento_007	
2026-08-21 21:33:05	RELEASE	evento_008	
2026-08-21 21:33:05	PRESS	evento_009	
2026-08-21 21:33:05	PRESS	evento_010	
2026-08-21 21:33:05	RELEASE	evento_011	
2026-08-21 21:33:05	PRESS	evento_012	
2026-08-21 21:33:05	RELEASE	evento_013	
2026-08-21 21:33:05	RELEASE	evento_014	
2026-08-21 21:33:05	PRESS	evento_015	
2026-08-21 21:33:06	RELEASE	evento_016	
2026-08-21 21:33:06	PRESS	evento_017	
2026-08-21 21:33:06	RELEASE	evento_018	
2026-08-21 21:33:06	PRESS	evento_019	
2026-08-21 21:33:06	RELEASE	evento_020	
2026-08-21 21:33:06	PRESS	evento_021	
2026-08-21 21:33:06	RELEASE	evento_022	
2026-08-21 21:33:06	PRESS	evento_023	
2026-08-21 21:33:06	RELEASE	evento_024	
2026-08-21 21:33:06	PRESS	evento_025	
2026-08-21 21:33:06	RELEASE	evento_026	



REGISTRO AUTOMÁTICO DE EVENTOS

8. CONCLUSIONES

El desarrollo permitió implementar un sistema capaz de registrar eventos del teclado durante la ejecución de un programa en Python.

Se logró cumplir con los criterios establecidos:

- Persistencia de información mediante un archivo local.
- Registro temporal mediante fecha y hora.
- Identificación individual de cada evento.
- Organización y almacenamiento de evidencias del funcionamiento.

La actividad permitió comprender el manejo de eventos, funciones automáticas y almacenamiento de datos en Python.